

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Student Performance Predictor

An AI/ML-based System to Predict Student Academic Outcomes

PROJECT REPORT

Submitted for: VITyarthi — Build Your Own Project (AIML)

Course	Artificial Intelligence & Machine Learning
Submission Type	Individual Project
Submitted by	Ritu Yadav
Registration No.	25MIB10064

2. Introduction

Academic institutions generate a large amount of data about student behaviour and performance -- attendance records, assignment completion, study patterns, and past grades. This project, **Student Performance Predictor**, applies supervised machine learning to this kind of data to forecast whether a student is likely to Pass or Fail, well before formal exam results are declared. The system was built end-to-end: from data preparation and storage, through model training and evaluation, to an interactive interface for real-time predictions.

The project directly applies core AIML course concepts: data preprocessing, feature engineering, supervised classification (Random Forest), model evaluation metrics, and data visualization, while also demonstrating good software engineering practice through modular code, a persistent data layer, logging, and automated tests.

3. Problem Statement

Teachers and academic mentors typically identify struggling students only after formal assessments -- at which point corrective action is delayed. There is a need for an early-warning system that uses ongoing, measurable indicators (study hours, attendance, assignment completion, past grades, sleep, extracurricular load, and parental support) to flag at-risk students proactively, so timely academic support can be offered.

4. Functional Requirements

The system implements three major functional modules, each with a clear input/output structure:

- **Data Management Module** -- Input: raw student records (generated or CSV). Output: a cleaned dataset persisted to a SQLite database.
- **ML Model Module** -- Input: cleaned feature set. Output: a trained classifier, evaluation metrics (accuracy, precision, recall, F1), and Pass/Fail predictions with a confidence score for new students.
- **Visualization & Reporting Module** -- Input: dataset + trained model metrics. Output: charts (class distribution, feature importance, confusion matrix, correlation heatmap) saved to disk.
- **CRUD on predictions** -- every prediction made is *created* (inserted) into the database and can be *read* back via the prediction-history view.

5. Non-Functional Requirements

Requirement	How it is addressed
Performance	Random Forest training on 800 records completes in well under a second; predictions are near-instant
Reliability	Data cleaning clips out-of-range values and fills missing data instead of crashing; stratified train/test sp
Security	No external network calls at runtime; all data stays local in the SQLite file, reducing exposure of studen
Usability	A simple numbered CLI menu guides the user through the entire workflow with sensible input defaults.
Scalability	The modular pipeline can scale to a larger dataset or be swapped to a different model with minimal co
Maintainability	Code is split into single-responsibility modules (config, logger, data, model, visualizer, app) with docstr
Error Handling	try/except blocks guard file and DB operations; invalid menu input is handled gracefully without crashin
Logging/Monitoring	A shared logger (src/logger.py) records every pipeline step to console and to outputs/app.log with time

6. System Architecture

The system follows a layered architecture with a clear separation between the presentation (CLI), application/logic, and data layers, as shown below.

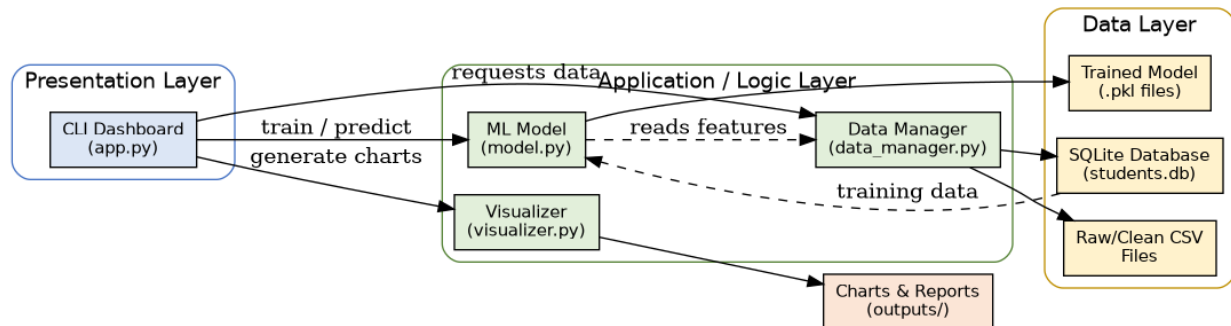


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

Two actors interact with the system: the Admin/Teacher (data prep, training, reports) and the Student/User (requesting a prediction, viewing history).

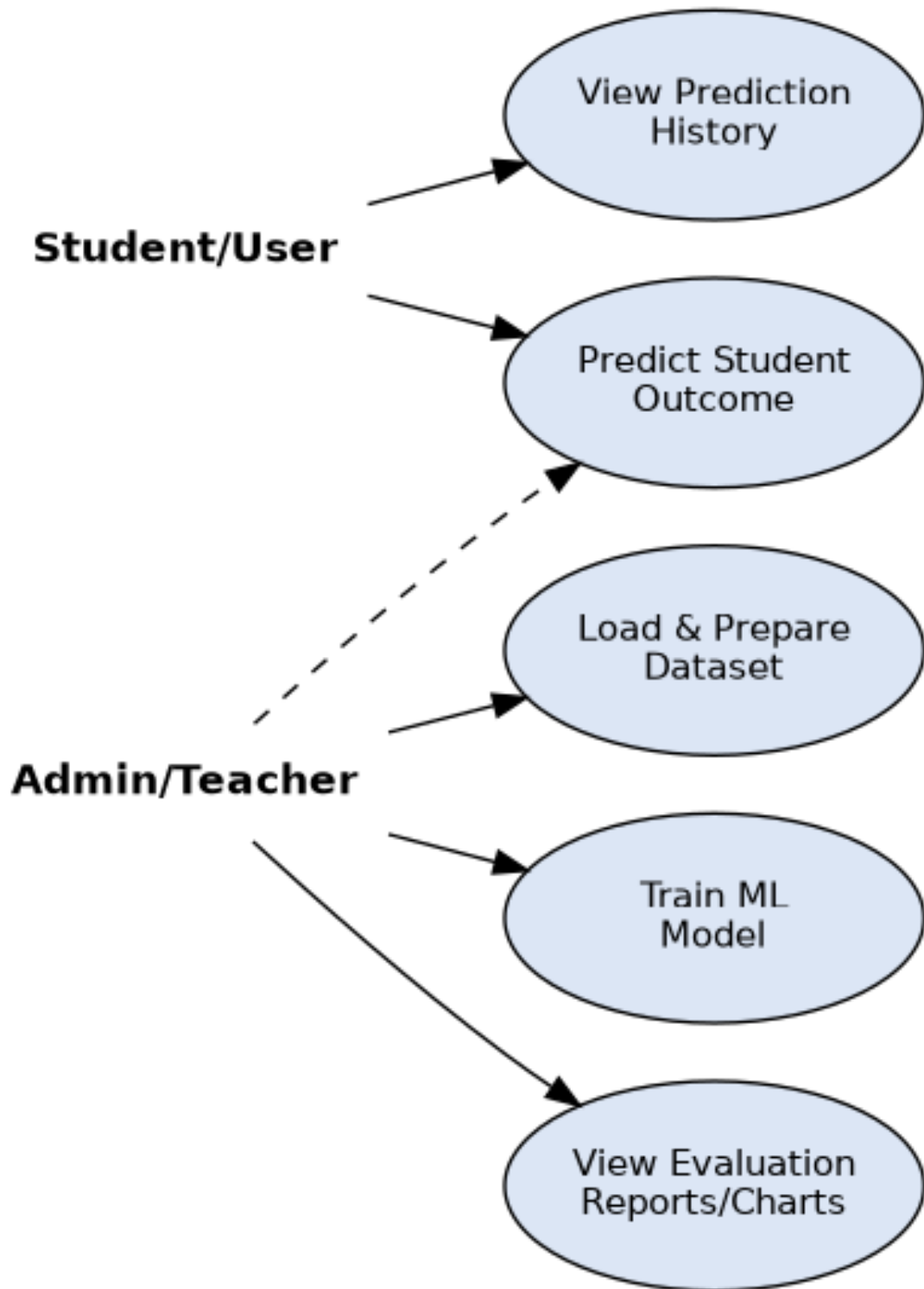


Figure 2: Use Case Diagram

7.2 Process Flow / Workflow Diagram

The end-to-end pipeline from raw data to a logged prediction is shown below.

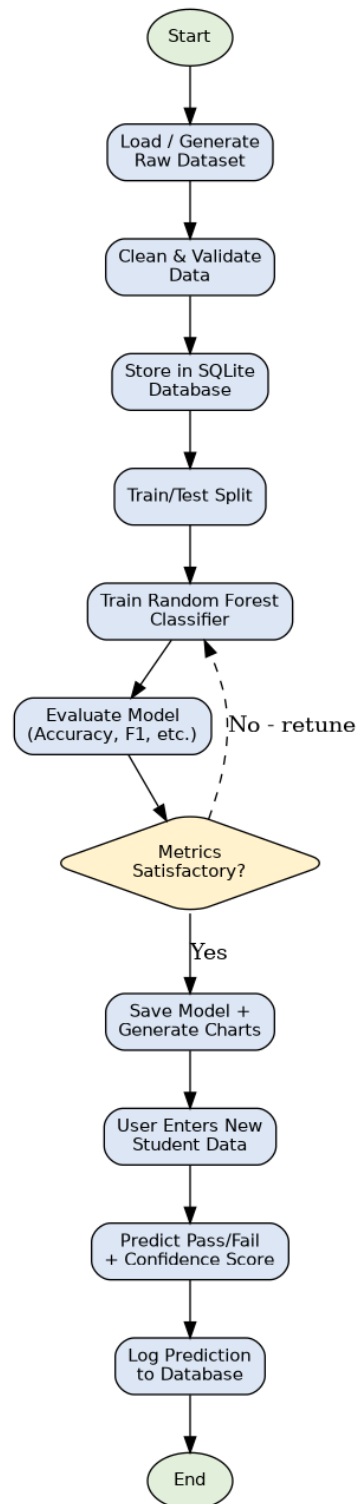


Figure 3: Process Flow / Workflow Diagram

7.3 Sequence Diagram

This diagram illustrates the runtime sequence of a single prediction request, from user input to the logged database record.

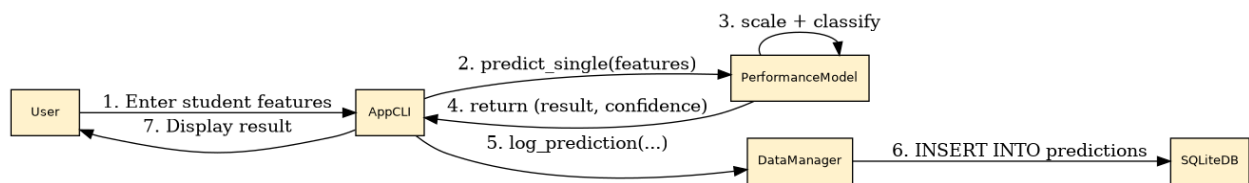


Figure 4: Sequence Diagram - Predict Outcome Flow

7.4 Class Diagram

The core classes/modules and their relationships are shown below.

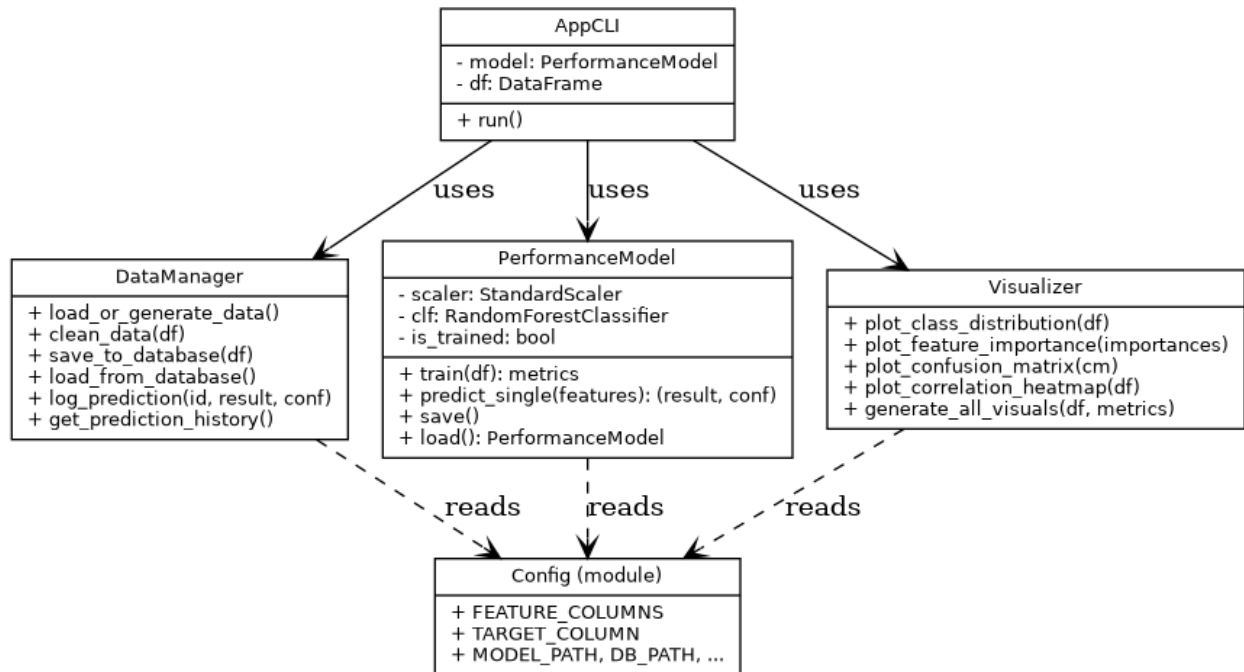


Figure 5: Class Diagram

7.5 ER Diagram (Database Schema)

The SQLite database has two tables: **students** (the training dataset) and **predictions** (a log of every prediction made, related to students by student_id).

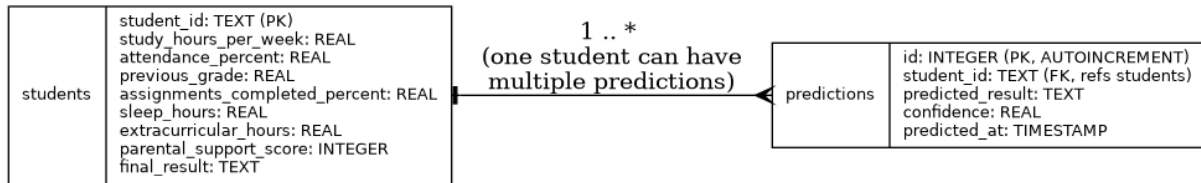


Figure 6: Entity-Relationship Diagram

8. Design Decisions & Rationale

- **Random Forest Classifier** was chosen over a single Decision Tree or Logistic Regression because it handles non-linear feature interactions well, is robust to outliers, and provides built-in feature-importance scores useful for explainability.
- **SQLite** was chosen as the storage layer because it requires no separate server process, keeps the project easy to run anywhere, and is sufficient for the dataset sizes involved, while still demonstrating real relational storage and CRUD operations.
- **StandardScaler** is applied before classification since Random Forest performance and, more importantly, comparability of feature magnitudes benefits from standardized inputs, and it keeps the pipeline ready for swapping in scale-sensitive models later.
- **A synthetic-but-realistic dataset** generator was used (rather than a bundled real dataset) to keep the project fully reproducible without external downloads, while the feature set and distributions were deliberately modeled on well-known, publicly documented student-performance datasets.
- **A CLI dashboard** (instead of a full web app) was chosen to keep the project focused on the AIML pipeline itself and easy to run/test in any environment without extra infrastructure, while still satisfying the 'clear user workflow' requirement.

9. Implementation Details

The project is implemented in Python 3, organized into the following modules:

File	Responsibility
src/config.py	Central configuration: paths, feature list, model hyperparameters.
src/logger.py	Shared logger writing to console and outputs/app.log.
src/data_manager.py	Dataset generation, cleaning/validation, SQLite storage, CRUD for predictions.
src/model.py	PerformanceModel class: scaling, training, evaluation, single-student prediction, save/load.
src/visualizer.py	Generates all evaluation/analytics charts (matplotlib/seaborn).
src/app.py	Interactive CLI dashboard tying all modules together.
main.py	Non-interactive script that runs the entire pipeline end-to-end.
tests/test_model.py	Automated unit tests (pytest) for data cleaning and model behaviour.
diagrams/gen_diagrams.py, gen_undo.py	Scripts that generate all design diagrams programmatically using Graphviz.

Dataset: 800 synthetic student records with 7 numeric/ordinal features (study_hours_per_week, attendance_percent, previous_grade,

assignments_completed_percent, sleep_hours, extracurricular_hours, parental_support_score) and a binary target (final_result: Pass/Fail). ~3% of records had a missing value intentionally introduced to exercise the cleaning logic.

Model selection rationale: Random Forest was compared conceptually against Logistic Regression and a single Decision Tree; Random Forest was selected for its balance of accuracy, robustness to noisy/correlated features, and interpretable feature-importance output, all relevant to an educational-intervention use case where explaining **why** a prediction was made matters.

Evaluation methodology: An 80/20 stratified train-test split preserves the Pass/Fail class ratio in both sets. The model is scored on accuracy, precision, recall, and F1-score (with "Pass" as the positive class), plus a confusion matrix for error-type analysis.

10. Screenshots / Results

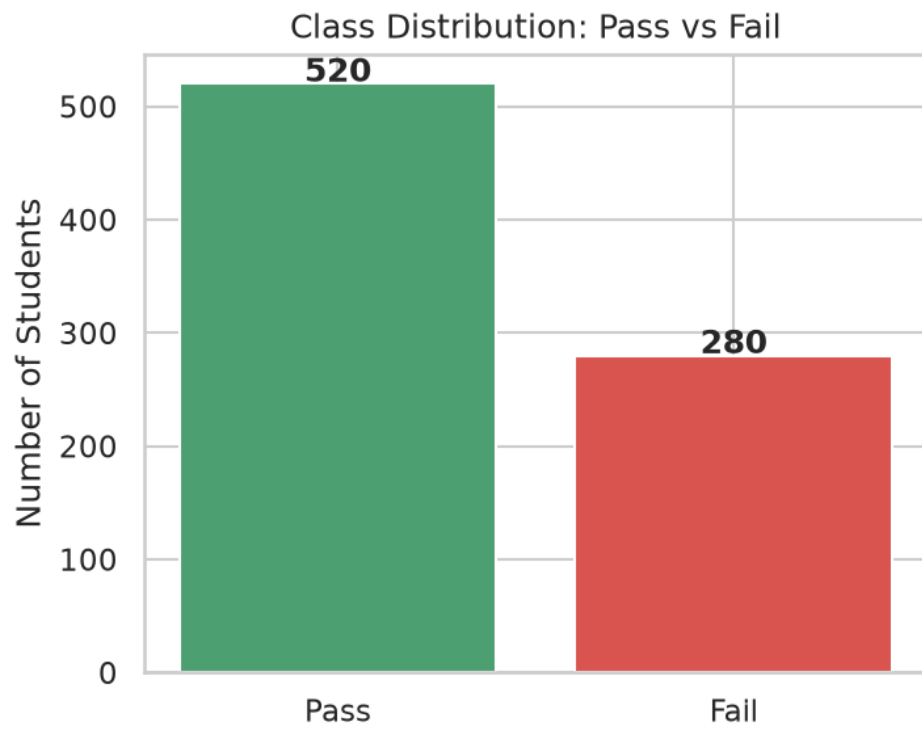


Figure 7: Class Distribution (Pass vs Fail)

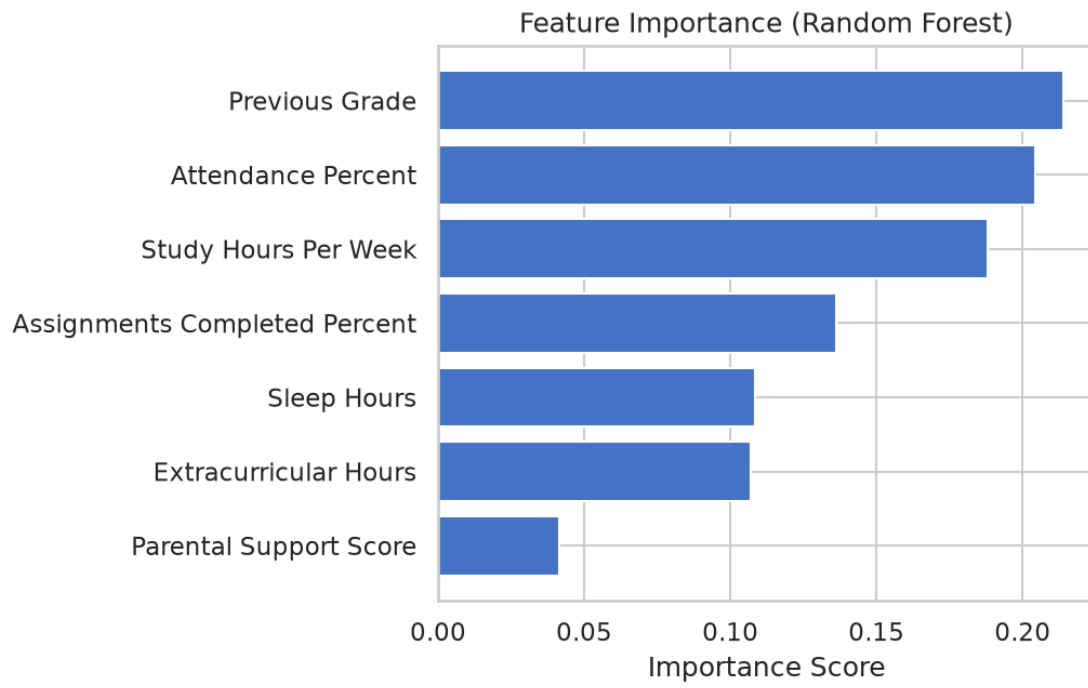


Figure 8: Feature Importance

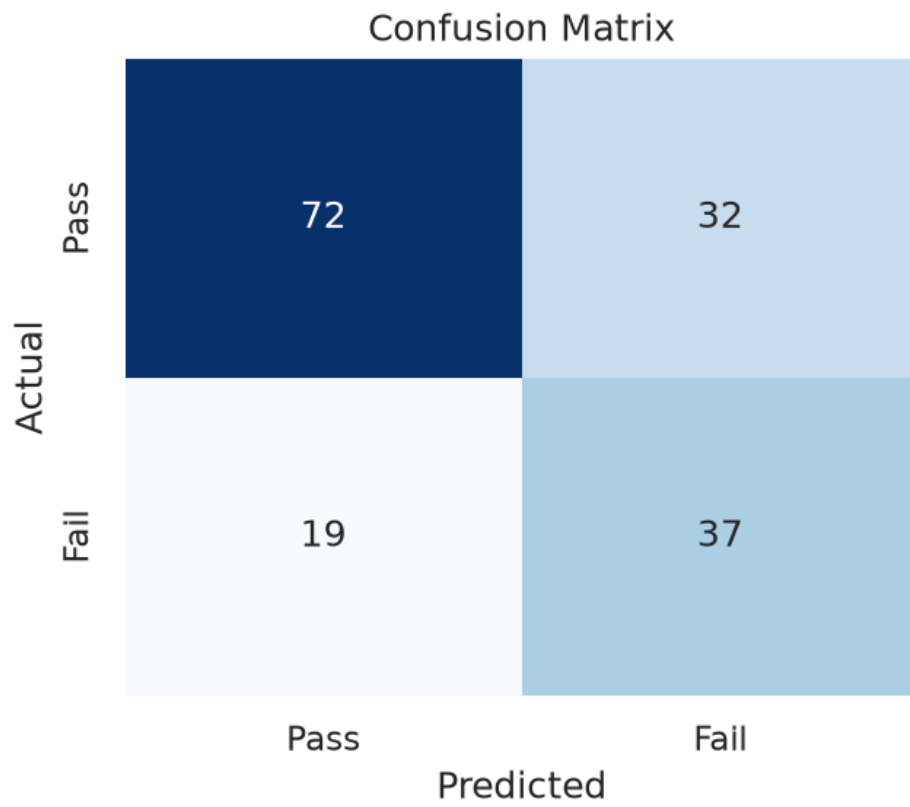


Figure 9: Confusion Matrix

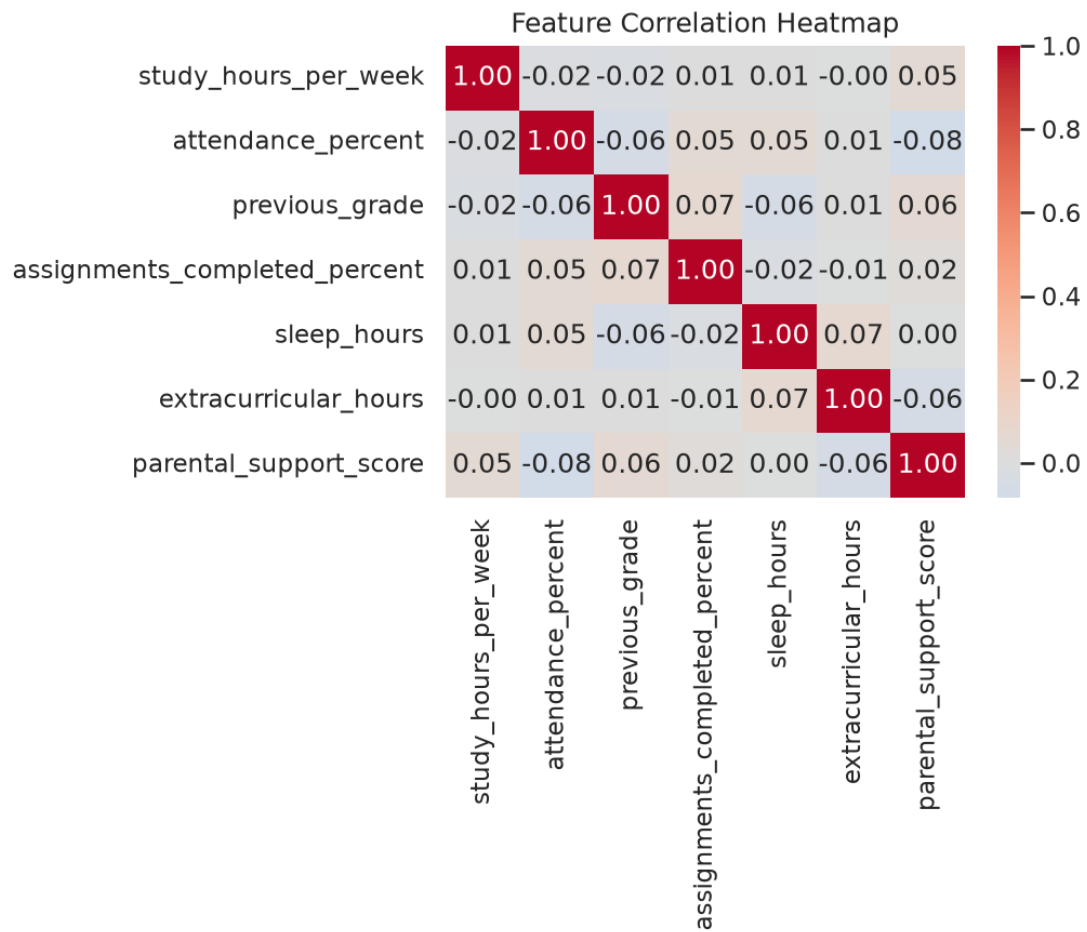


Figure 10: Feature Correlation Heatmap

11. Testing Approach

Automated unit tests were written using **pytest** and cover both the data pipeline and the ML model:

- Dataset generation produces the expected shape and required columns.
- Generated target values are restricted to the valid label set (Pass/Fail).
- Cleaning removes all missing values from the feature columns.
- Cleaning enforces valid value ranges (e.g., percentages within 0-100).
- The model trains successfully and returns metrics within valid bounds (0-1).
- Single-student prediction returns a valid label and a confidence score in [0, 1].

All 6 tests pass consistently (*pytest tests/ -v*). Manual end-to-end testing was also performed by running *main.py* and the interactive *src/app.py* dashboard through every menu option.

12. Challenges Faced

- **Balancing dataset realism vs. separability** -- an early version of the synthetic data generator produced an almost perfectly separable Pass/Fail boundary, giving unrealistically high (~99%) accuracy. Noise was added to the underlying scoring formula to make the classification problem more realistic and representative of real-world data.
- **Keeping the diagrams reproducible** -- rather than hand-drawing UML diagrams in an external tool (which would be hard to version-control and regenerate), all six diagrams were generated programmatically with Graphviz, so they stay in sync with the code.
- **Designing a clean CRUD flow** for predictions without over-engineering the storage layer -- solved by adding a lightweight *predictions* table alongside the main *students* table in the same SQLite database.

13. Learnings & Key Takeaways

- Hands-on experience with the full supervised ML lifecycle: data cleaning, feature scaling, model training, evaluation metrics, and deploying a model for real-time single-record inference.

- Practical understanding of why realistic (noisy) data is essential to building and evaluating a meaningful classifier, rather than an artificially perfect one.
- Experience integrating an ML pipeline with a persistent relational data layer (SQLite) and CRUD operations, not just a one-off notebook.
- Reinforced good software engineering habits -- modular design, logging, configuration management, and automated testing -- applied specifically to an ML project.

14. Future Enhancements

- Replace the synthetic dataset with a real, anonymized institutional dataset (with appropriate consent/privacy safeguards).
- Add a web-based dashboard (e.g., Flask/Streamlit) for a richer, multi-user interface.
- Experiment with additional models (Gradient Boosting, XGBoost, Neural Networks) and hyperparameter tuning to further improve accuracy.
- Add authentication and role-based access so teachers and students see appropriately scoped views.
- Extend the target from binary Pass/Fail to a multi-class grade-band prediction.

15. References

- Cortez, P. and Silva, A. (2008). *Using Data Mining to Predict Secondary School Student Performance*. UCI Machine Learning Repository -- Student Performance Data Set (feature-distribution reference for the synthetic dataset).
- Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
- Official documentation: scikit-learn.org, pandas.pydata.org, sqlite.org, graphviz.org.